

## CSCE 641 COMPUTER GRAPHICS · MODULE 10 · LECTURE NOTES

# Shadows

*Visibility from the light's point of view, and everything that goes wrong with it.*

## MODULE OUTCOMES

- Implement basic shadow mapping end to end.
- Diagnose acne, peter-panning, and aliasing, and explain the cause of each.
- Apply slope-scaled bias and normal offset correctly.
- Implement PCF and explain what it does and does not fix.
- Explain cascaded shadow maps and why directional lights need them.

## 1 Shadows as a visibility problem

A point is lit if there is an unobstructed path between it and the light. That is a visibility query, and you have already built a machine that answers visibility queries: the depth buffer. The only change is where the camera is.

Render the scene from the light's position, storing depth only. The resulting buffer records, for every direction the light looks, how far away the nearest surface is — that is, the surface the light illuminates. Then, during normal rendering, transform each fragment into the light's clip space and compare: if the fragment is further from the light than the stored depth, something is in between, and it is in shadow.

```
// Pass 1: from the light, depth only.
glViewport(0, 0, SHADOW_W, SHADOW_H);
glBindFramebuffer(GL_FRAMEBUFFER, shadowFBO);
glClear(GL_DEPTH_BUFFER_BIT);
draw_scene(lightView, lightProj);      // no colour, no shading

// Pass 2: from the camera, with the lookup.
float shadow(vec3 worldPos) {
    vec4 lp = uLightVP * vec4(worldPos, 1.0);
    vec3 pc = lp.xyz / lp.w;
    pc = pc * 0.5 + 0.5;                // NDC -> texture coordinates

    if (pc.z > 1.0) return 1.0;        // beyond the light's far plane: lit

    float stored = texture(uShadowMap, pc.xy).r;
    return (pc.z - BIAS) > stored ? 0.0 : 1.0;
}
```

The technique is image-space and geometry-independent: it does not care how the scene is built, handles arbitrary geometry including alpha-tested foliage, and costs one extra pass. It is, for these reasons, essentially universal. Everything difficult about it is in the artifacts.

## 2 Acne

The first thing you will see is striped self-shadowing: a moiré-like pattern of dark bands across surfaces that should be fully lit.

The cause is a resolution mismatch. The shadow map stores one depth per texel, but that texel corresponds to an *area* of the receiving surface. If the surface is sloped relative to the light, the true depth varies continuously across that area while the stored value is a single number sampled somewhere within it. Points on the far side of the sample are deeper than the stored depth and therefore test as shadowed — by themselves.

### THE DIAGNOSTIC CLUE

Acne is worst on surfaces oblique to the light and absent on surfaces facing it directly.

That is because the depth range spanned by one texel is proportional to the surface slope: a perpendicular surface has constant depth within the texel, so there is nothing to misjudge.

This immediately tells you the correction should scale with slope.

### 2.1 Bias

Offset the comparison so a surface must be meaningfully deeper than the stored value before it counts as shadowed. A constant bias works, badly: the value large enough to suppress acne on steep surfaces is large enough to detach shadows on flat ones.

$$\text{bias} = \max( c_2 \cdot (1 - n \cdot l), c_1 )$$

Slope-scaled bias makes the offset proportional to the surface slope, which is proportional to the depth range inside a texel. A small constant floor handles the perpendicular case. This removes most of the dilemma.

### 2.2 Normal offset

A frequently better alternative: rather than biasing the depth comparison, move the *sample position* along the surface normal before projecting into light space.

$$p' = p + n \cdot k \quad (k \text{ scaled by texel world size and slope})$$

This translates the sample in world space, perpendicular to the surface, rather than pushing it along the light ray. It tends to produce less peter-panning for the same acne suppression, because it does not move the contact point away from the floor in the direction that matters. In practice a combination of a small slope-scaled depth bias and a normal offset works best.

### 3 Peter-panning

---

Over-correct the bias and shadows detach from their objects: the contact shadow where an object meets the floor is pushed out of shadow, so the object appears to hover. The name is self-explanatory once you have seen it.

A partial fix is to render only **back faces** into the shadow map. The surfaces being biased are then ones the camera never sees, so the bias cannot detach a visible contact point. This works well for closed watertight geometry and fails for open sheets — a single-sided wall, a leaf card — which have no back faces to render. Most engines therefore do it selectively.

## 4 Shadow aliasing and PCF

Shadow edges are blocky because the shadow map is a finite sampling of the depth function, and one shadow texel frequently covers many screen pixels. This is exactly Module 09's phenomenon with the shadow map as the sampling grid, and the same two strategies apply: sample more, or filter.

### 4.1 Percentage-closer filtering

#### FILTER THE RESULTS, NOT THE DEPTHS

The obvious approach — average the depths in a neighbourhood, then compare once — does not work. Comparison is a step function, so comparing a single averaged depth still yields a hard 0 or 1. You get a slightly wrong hard edge instead of a soft one.

PCF compares *first*, at each tap, and averages the binary results. Nine taps give ten possible values between 0 and 1, which is a gradient. This ordering is the entire content of the technique, and it is easy to implement backwards.

```
float pcf(vec3 pc, float bias) {
    float sum = 0.0;
    vec2 texel = 1.0 / vec2(textureSize(uShadowMap, 0));
    for (int y = -1; y <= 1; ++y)
        for (int x = -1; x <= 1; ++x) {
            float d = texture(uShadowMap, pc.xy + vec2(x,y)*texel).r;
            sum += (pc.z - bias) > d ? 0.0 : 1.0;    // compare, THEN accumulate
        }
    return sum / 9.0;
}
```

Hardware comparison samplers (sampler2DShadow) do the compare-then-filter in one instruction with bilinear weighting, which is both faster and smoother than a manual loop. Use them.

### 4.2 PCF is blur, not penumbra

A real soft shadow has a penumbra whose width grows with the distance between occluder and receiver: the shadow under your foot is sharp, the shadow of a cloud is diffuse. PCF applies a *constant* filter width everywhere, so it produces uniformly soft edges regardless of geometry. It looks better than a hard edge and it is not physically meaningful.

Percentage-closer soft shadows (PCSS) adds a blocker-search step: sample the shadow map to estimate the average depth of occluders, compute the penumbra width that geometry implies, and size the PCF kernel accordingly. More correct, several times the cost, and still an approximation — but a convincing one.

## 5 Cascaded shadow maps

A directional light has no position, so its shadow map must cover the entire visible scene. For an outdoor view that might be one metre to one kilometre. A single 2048×2048 map spread over a kilometre gives roughly half a metre per texel, which is useless for anything near the camera.

The mismatch is not uniform: nearby geometry occupies many screen pixels per shadow texel, distant geometry occupies few. This is **perspective aliasing**, and the fix is to match shadow resolution to screen resolution by splitting the problem.

- Divide the camera frustum into 3–4 depth slices.
- Render a separate shadow map fitted tightly to each slice.
- In the fragment shader, select the cascade by view depth.
- Blend across cascade boundaries, or a hard line appears across the scene at a fixed distance.
- **Snap each cascade to shadow-texel boundaries** as the camera moves. Without this the shadow map's sampling grid slides continuously relative to the world, and shadow edges crawl and shimmer — a distinctive, very visible artifact.

Split placement matters. Pure logarithmic splits are theoretically right for perspective aliasing but waste resolution at the far end; uniform splits waste it at the near end. Practical schemes blend the two with a tunable weight, and then let an artist adjust it.

## 6 By light type

| Light             | Shadow projection                       | Cost                     | Notes                                                                                                                    |
|-------------------|-----------------------------------------|--------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Directional (sun) | Orthographic, one per cascade           | 3–4 scene renders        | Fit each cascade to its frustum slice; snap to texels.                                                                   |
| Spot              | Perspective matching the cone           | 1 scene render           | The simplest case. The light frustum is the cone.                                                                        |
| Point             | Six perspective renders into a cube map | 6 scene renders          | Expensive, which is why engines limit shadow-casting point lights severely. Layered rendering can reduce it to one pass. |
| Area              | No exact method                         | Stochastic or ray-traced | Shadow maps assume a point source. Area lights need many samples, or ray tracing.                                        |

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                     | What it covers for this module                                                                                                |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">GAMES101 Lecture 10 — Shadow Mapping</a>                         | The algorithm and the bias problem, clearly derived.                                                                          |
| <a href="#">LearnOpenGL — Shadow Mapping and Point Shadows</a>               | Working implementation including the cube-map point-light case.                                                               |
| <a href="#">Microsoft — 'Common Techniques to Improve Shadow Depth Maps'</a> | The definitive free reference on cascades, bias strategies, and texel snapping. Read it in full before implementing cascades. |
| <a href="#">Fernando — Percentage-Closer Soft Shadows (NVIDIA, free)</a>     | The original PCSS formulation with the blocker-search derivation.                                                             |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement basic shadow mapping for a single spot light. Capture the result with zero bias and observe acne in its natural state.
- 2 Implement constant bias, then slope-scaled bias, then normal offset. Capture a scene with both steep and flat surfaces under each, and find the constant-bias value that trades acne for peter-panning. Document that there is no good value.
- 3 Implement 3×3 PCF manually, then switch to a hardware comparison sampler. Measure both and compare quality.
- 4 Deliberately implement PCF backwards — average the depths, then compare — and capture the result. Explain in one sentence why it produces a hard edge.
- 5 Implement 3-cascade shadow maps for a directional light. First without texel snapping: move the camera slowly and record the crawling. Then add snapping and record again.
- 6 Visualise your cascade selection by tinting each cascade a different colour. Use it to check your split distances are sensible for your scene.

## Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Explain shadow mapping in two sentences, and say which earlier module pass 1 reuses unchanged.
- 2 What causes shadow acne, and why is it worst on surfaces oblique to the light?
- 3 Why is a constant bias a losing proposition? What two techniques break the dilemma, and how does each work?

- 4 Why must PCF compare before averaging? What happens if you reverse the order?
- 5 Explain why PCF does not produce physically correct soft shadows, and what PCSS adds.
- 6 Why do directional lights need cascades? What goes wrong if cascades are not snapped to texel boundaries?